A SELF-SUPERVISED BEHAVIORAL GRAPH FRAMEWORK FOR ZERO-DAY CYBER ATTACK DETECTION WITH CONTINUAL LEARNING

A PROJECT REPORT

Submitted by

KAVISH S - 2022115001
SHREYAS S - 2022115046
SASHANK G - 2022115051
PHAVANKUMAR RL - 2022115054

to the Faculty of

INFORMATION AND COMMUNICATION ENGINEERING

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

ANNA UNIVERSITY : CHENNAI 600 025

FEBRUARY 2026

ANNA UNIVERSITY

CHENNAI – 600 025

BONA FIDE CERTIFICATE

Certified that this project report titled **"A SELF-SUPERVISED BEHAVIORAL GRAPH FRAMEWORK FOR ZERO-DAY CYBER ATTACK DETECTION WITH CONTINUAL LEARNING"** is the bona fide work of **KAVISH S (2022115001)**, **SHREYAS S (2022115046)**, **SASHANK G (2022115051)**, and **PHAVANKUMAR RL (2022115054)** who carried out the project work under my supervision.

Certified further that to the best of my knowledge and belief, the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion for this or any other candidate.

PLACE: CHENNAI

DATE: _____

Dr. K. VIDYA
ASSOCIATE PROFESSOR
PROJECT GUIDE
DEPARTMENT OF IST, CEG
ANNA UNIVERSITY
CHENNAI – 600 025

COUNTERSIGNED

CONTENTS

1 Introduction **1**

1.1 BACKGROUND AND MOTIVATION 1

1.2 CHALLENGES 1

1.2.1 LACK OF LABELED ATTACK DATA 1

1.2.2 HETEROGENEOUS SYSTEM BEHAVIOR 1

1.2.3 DYNAMIC AND NOISY ENVIRONMENTS 2

1.3 PROBLEM STATEMENT 2

1.4 PROPOSED SOLUTION 2

1.4.1 GRAPH-BASED BEHAVIORAL MODELING 2

1.4.2 SELF-SUPERVISED GRAPH AUTOENCODING 2

1.4.3 HYBRID ANOMALY SCORING 2

1.5 OBJECTIVES 3

1.6 SCOPE OF THE PROJECT 3

1.7 ORGANIZATION OF THE REPORT 3

2 Literature Survey **4**

2.1 OVERVIEW 4

2.2 DATA COLLECTION AND SYSTEM BEHAVIOR MODELING 4

2.3 BEHAVIORAL GRAPH CONSTRUCTION MODULE 5

2.3.1 HETEROGENEOUS GRAPH MODELING 5

2.4 GRAPH NEURAL NETWORK ENCODER 6

2.4.1 GRAPH CONVOLUTIONAL NETWORKS 6

2.5 GRAPH AUTOENCODER AND SELF-SUPERVISED LEARNING 7

2.5.1 SELF-SUPERVISED LEARNING PRINCIPLE 7

2.5.2 GRAPH AUTOENCODER 7

2.6 ANOMALY DETECTION AND STATISTICAL THRESHOLDING 7

2.6.1 RECONSTRUCTION-BASED ANOMALY DETECTION 7

2.7 ATTACK SIMULATION FRAMEWORK 8

2.8 EXPLAINABILITY AND GRAPH VISUALIZATION 8

2.9 CONTINUAL LEARNING AND ADAPTIVE DETECTION 9

2.10 SUMMARY 9

3 System Design **10**
3.1 SYSTEM OVERVIEW 10
3.2 SYSTEM ARCHITECTURE 10
3.3 MODULE DESCRIPTIONS 12
3.3.1 SYSTEM MONITORING AND DATA COLLECTION 12
3.3.2 BEHAVIORAL GRAPH CONSTRUCTION 13
3.3.3 GRAPH AUTOENCODER TRAINING AND BASELINE 14
3.3.4 DETECTION AND ALERTING 15
3.3.5 ATTACK SIMULATION AND EVALUATION 15
3.3.6 VISUALIZATION AND REPORTING 16
3.4 END-TO-END DATA FLOW 16
3.5 SUMMARY 17

REFERENCES **18**

CHAPTER 1 INTRODUCTION

1.1 Background and Motivation

Modern operating systems generate a continuous stream of low-level events such as process creation, file access, and network connections. These events capture how a system behaves over time and can be used to identify malicious activity. Traditional signature-based intrusion detection systems are effective for known threats but are unable to recognize new attack patterns. As attack techniques evolve rapidly, there is a strong need for zero-day detection methods that do not depend on labeled attack data.

Graph-based representations provide an effective way to capture relationships among system entities (processes, files, and sockets). In parallel, self-supervised learning has emerged as a promising approach for modeling normal behavior without explicit labels. This project integrates these directions to build a practical zero-day attack detection pipeline that learns from normal activity and flags deviations as anomalies.

1.2 Challenges

Despite the promise of data-driven security, several challenges arise in real-time host-based detection.

1.2.1 Lack of Labeled Attack Data

Collecting labeled attack traces for every new threat is expensive and often impossible. A practical system must operate without attack labels while still achieving useful detection performance.

1.2.2 Heterogeneous System Behavior

System activity involves multiple entity types and interactions. Modeling only individual events ignores the structural relationships that distinguish normal workflows from attacks.

1.2.3 Dynamic and Noisy Environments

Normal system behavior varies over time depending on user activity, installed software, and background services. This variability introduces noise and increases the risk of false positives.

1.3 Problem Statement

The central problem is to design a host-based detection system that can identify zero-day attacks without relying on labeled attack data. The system must capture complex behavioral relationships among system entities, learn normal patterns from real-time monitoring, and detect anomalous behavior while remaining lightweight enough for practical deployment.

1.4 Proposed Solution

This project proposes a self-supervised behavioral graph-based detection system. It monitors real system activity, constructs heterogeneous graphs, and trains a graph autoencoder to learn normal behavior. During testing, a hybrid anomaly detector combines reconstruction error with structural features to detect suspicious activity. Simulated attack scenarios are used to evaluate detection performance in a controlled environment.

1.4.1 Graph-Based Behavioral Modeling

Events are converted into graphs with PROCESS, FILE, and SOCKET nodes and edges representing interactions such as executes, reads, writes, spawns, and connects. This representation captures the context of system activity rather than isolated event counts.

1.4.2 Self-Supervised Graph Autoencoding

A graph autoencoder learns latent representations of normal graphs and attempts to reconstruct their structure. Graphs that deviate significantly from learned patterns produce high reconstruction errors and are flagged as anomalies.

1.4.3 Hybrid Anomaly Scoring

Anomaly scores are computed using both reconstruction error and structural statistics such as node-type ratios, degree distributions, and edge density. This hybrid approach improves robustness compared to using reconstruction error alone.

1.5 Objectives

The objectives of this project are:

1. Collect real-time system events for processes, files, and network connections.
2. Construct heterogeneous temporal graphs capturing behavioral relationships among system entities.
3. Train a self-supervised graph autoencoder to learn normal behavior without attack labels.
4. Detect anomalies using combined reconstruction and structural features.
5. Simulate representative attacks to evaluate detection performance.
6. Generate visualizations and logs to support analysis and reporting.

1.6 Scope of the Project

In Scope: real-time host monitoring, behavioral graph construction, self-supervised graph autoencoding, hybrid anomaly detection, simulated attack scenarios (reverse shell, privilege escalation, data exfiltration), performance metrics, and visualization of results.

Out of Scope: kernel-level instrumentation, enterprise-scale distributed deployment, full MITRE ATT&CK mapping, and automated incident response. These can be addressed in later phases as extensions.

1.7 Organization of the Report

The chapters in this report are organized as follows:

CHAPTER 1 INTRODUCTION presents the problem, challenges, and objectives of the project.

CHAPTER 2 LITERATURE SURVEY reviews related work in self-supervised anomaly detection, graph-based intrusion detection, and continual learning.

CHAPTER 3 SYSTEM DESIGN describes the system architecture and module design of the proposed solution.

Subsequent chapters cover implementation, experiments, results, and conclusions.

CHAPTER 2 LITERATURE SURVEY

2.1 Overview

Zero-day attack detection requires identifying malicious behavior that has never been seen before and does not have known signatures. Traditional signature-based systems fail in such scenarios because they rely on predefined attack patterns. Therefore, anomaly-based detection, self-supervised representation learning, and graph-based modeling have emerged as promising directions.

This literature survey explains the research foundations of each major module in the proposed system:

1. Data Collection and System Behavior Modeling
2. Behavioral Graph Construction
3. Graph Neural Network Encoder
4. Graph Autoencoder (Self-Supervised Learning)
5. Anomaly Detection and Statistical Thresholding
6. Attack Simulation Framework
7. Explainability and Visualization
8. Continual Learning for Adaptive Detection

Each module is supported by established research in deep learning and cybersecurity.

2.2 Data Collection and System Behavior Modeling

The foundation of the proposed system lies in modeling operating system behavior through system calls. System calls act as the interface between user-space applications and the kernel. Every process execution, file access, network connection, or privilege change is recorded as a syscall event such as `execve`, `fork`, `open`, `read`, `write`, or `connect`.

Earlier intrusion detection systems relied on sequential modeling of system calls using Hidden Markov Models or Recurrent Neural Networks. However, these

approaches capture only sequential patterns and ignore structural relationships between entities.

Recent research suggests that representing system interactions as structured graphs preserves richer contextual information. Instead of treating syscalls as isolated sequences, they can be mapped to relationships between entities:

- Process → executes → file
- Process → reads/writes → file
- Process → connects → socket
- Process → spawns → process

This structured representation aligns with anomaly-based intrusion detection principles discussed in modern cybersecurity surveys. Learning normal behavior from syscall-derived graphs allows the system to detect deviations without requiring attack labels. Thus, the Data Collection module is grounded in behavioral modeling literature and anomaly-based IDS research.

2.3 Behavioral Graph Construction Module

Raw syscall logs must be transformed into structured graphs suitable for neural processing. The system constructs heterogeneous behavioral graphs using fixed time windows (e.g., 30 seconds). Each window forms a snapshot graph representing system activity during that period.

2.3.1 Heterogeneous Graph Modeling

System behavior involves multiple entity types:

- PROCESS nodes
- FILE nodes
- SOCKET nodes

Edges represent semantic relationships:

- EXECUTES
- READS
- WRITES
- SPAWNS

- CONNECTS

Research on heterogeneous graph neural networks demonstrates that modeling multiple node and edge types improves representation quality when different relationship semantics exist. Unlike homogeneous graphs, heterogeneous graphs preserve role-specific information and interaction context.

Time-windowed graphs also connect to temporal graph learning literature, where dynamic system behavior is represented as evolving graph snapshots. This allows detection of behavioral deviations over time. Thus, the Behavioral Graph Construction module is supported by research in heterogeneous graph representation learning and dynamic graph modeling.

2.4 Graph Neural Network Encoder

Once behavioral graphs are constructed, they must be encoded into meaningful vector representations. This is achieved using Graph Convolutional Networks (GCNs).

2.4.1 Graph Convolutional Networks

GCNs extend convolution operations to graph-structured data. Instead of sliding filters over grids (like images), GCNs aggregate feature information from neighboring nodes. Mathematically, a GCN layer performs:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}\right)$$

where:

- $\tilde{A}$ is the adjacency matrix with self-loops
- $\tilde{D}$ is the degree matrix
- $H^{(l)}$ is the node feature matrix
- $W^{(l)}$ is the learnable weight matrix
- σ is a non-linear activation

In this system, a two-layer GCN encoder maps node features from input dimension to 64 hidden units and then to 32-dimensional embeddings. GCNs preserve topological structure while learning compact representations. They are particularly effective in capturing neighborhood-level structural patterns, which is essential for identifying abnormal process-file or process-socket relationships.

Attention-based graph models, such as Graph Attention Networks, provide future scope for adaptive neighbor weighting but are not part of the core implementation.

Thus, the Graph Encoder module is grounded in foundational graph neural network research.

2.5 Graph Autoencoder and Self-Supervised Learning

The central innovation of the system lies in self-supervised learning using a Graph Autoencoder.

2.5.1 Self-Supervised Learning Principle

Unlike supervised learning, self-supervised models do not require labeled attack data. Instead, they learn representations by solving a proxy task. In this system, the proxy task is reconstructing the graph adjacency matrix.

2.5.2 Graph Autoencoder

The architecture consists of:

- Encoder: Two-layer GCN
- Decoder: Inner-product reconstruction

The decoder reconstructs adjacency as:

$$\hat{A} = \sigma(ZZ^\top)$$

where Z is the node embedding matrix. The training objective minimizes binary cross-entropy between original adjacency matrix A and reconstructed matrix $\hat{A}$. This generative self-supervised approach ensures that the model learns structural patterns of normal system behavior. If a new graph significantly deviates from learned normal patterns, reconstruction error increases. This module is supported by literature on variational graph autoencoders and unsupervised graph representation learning.

2.6 Anomaly Detection and Statistical Thresholding

Once reconstruction error is computed, it must be converted into a detection decision.

2.6.1 Reconstruction-Based Anomaly Detection

Research in unsupervised anomaly detection shows that normal samples reconstruct well, while anomalies produce higher reconstruction error. This principle has been widely used in autoencoder-based anomaly detection. For each graph:

$$AnomalyScore = A - \hat{A}$$

To determine whether a graph is anomalous, the system computes:

$$Threshold = \mu + 3\sigma$$

where μ is the mean reconstruction error of normal training graphs and σ is the standard deviation. This Z-score based statistical thresholding ensures reproducibility and eliminates arbitrary tuning. It aligns with statistical anomaly detection literature. Additionally, structural anomaly detection methods such as Isolation Forest provide complementary approaches for hybrid scoring. Thus, the Anomaly Detection module is rooted in both deep autoencoding anomaly detection and classical statistical methods.

2.7 Attack Simulation Framework

Evaluating zero-day detection requires generating realistic adversarial behavior. The system incorporates attack patterns inspired by the MITRE ATT&CK framework, which categorizes adversarial techniques such as:

- Reverse Shell
- Privilege Escalation
- Data Exfiltration

These attack simulations create distinctive graph structures that deviate from normal system behavior. For example, reverse shells create abnormal outbound socket connections, privilege escalation introduces unexpected process spawning chains, and exfiltration produces high-frequency file access followed by network transmission. By mapping simulated attacks to known adversarial tactics, the system ensures realistic evaluation scenarios.

2.8 Explainability and Graph Visualization

Security systems must provide interpretability to analysts. Graph visualization tools such as NetworkX allow:

- Node coloring by type
- Edge labeling by interaction
- Hierarchical layout for clarity

Explainable AI research in cybersecurity emphasizes the importance of identifying which relationships triggered anomaly detection. In future extensions, feature attribution techniques can highlight anomalous subgraphs. Thus, visualization supports analyst trust and operational usability.

2.9 Continual Learning and Adaptive Detection

A major challenge in intrusion detection is concept drift, where normal behavior evolves over time. Static models degrade in performance. Continual learning research proposes methods such as Elastic Weight Consolidation (EWC) to prevent catastrophic forgetting while adapting to new data. In this project's future phase:

- The model will update using new normal behavior
- Important weights will be regularized
- Replay buffers may preserve historical behavior samples

This ensures long-term adaptability without losing previously learned patterns.

2.10 Summary

The literature demonstrates that combining heterogeneous graph modeling, graph autoencoders, and statistical anomaly detection provides a strong foundation for zero-day attack detection. Self-supervised learning eliminates dependency on labeled attack data, making the system practical for real-world deployment. Each module in the proposed architecture is grounded in validated research:

- Behavioral modeling $\rightarrow$ anomaly-based IDS literature
- Graph construction $\rightarrow$ heterogeneous GNN research
- Graph encoding $\rightarrow$ GCN theory
- Self-supervised reconstruction $\rightarrow$ graph autoencoder research
- Anomaly scoring $\rightarrow$ unsupervised detection methods
- Thresholding $\rightarrow$ statistical Z-score techniques
- Adaptation $\rightarrow$ continual learning research

Together, these components form a technically rigorous and research-backed framework for zero-day intrusion detection.

CHAPTER 3 SYSTEM DESIGN

3.1 System Overview

The system is designed as a modular pipeline that learns normal host behavior and flags abnormal behavior without relying on pre-labeled attacks. It continuously collects host activity, converts it into behavioral graphs, trains a graph autoencoder on normal graphs, and then scores new graphs for anomalies. The design emphasizes clear boundaries between modules so that data collection, graph construction, learning, detection, and evaluation can be improved independently.

3.2 System Architecture

Figure 3.1 presents the end-to-end architecture. It shows how raw events flow into graph construction, how the model learns normal patterns, and how alerts are generated and visualized. The diagram also highlights the evaluation loop where simulated attacks are injected to test detection.

The figure summarizes the full pipeline and the feedback loop that enables evaluation of detection performance.

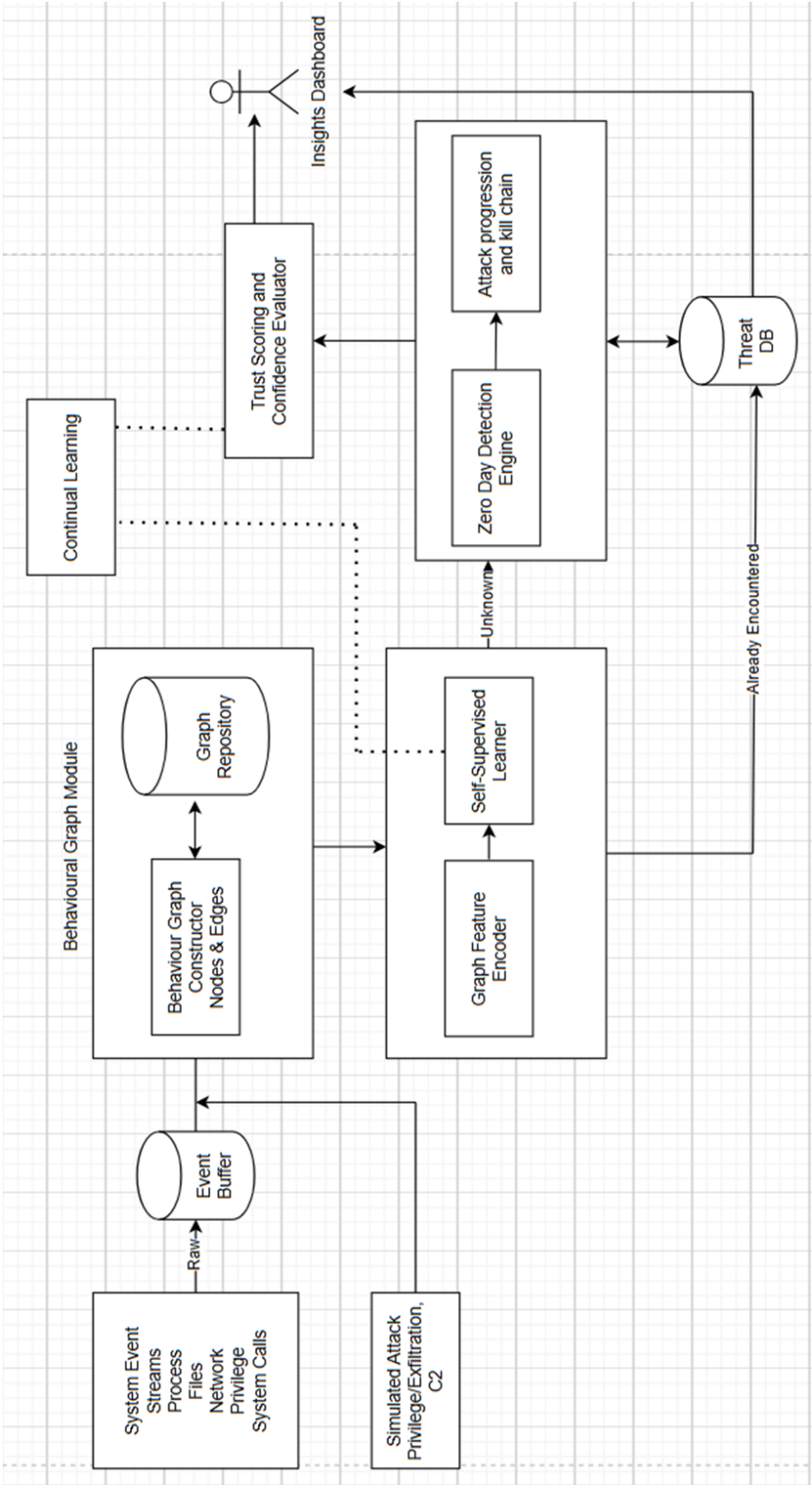

Figure 3.1: Overall system architecture for the zero-day detection pipeline.

This architecture view provides the reference flow for the module descriptions that follow.

3.3 Module Descriptions

3.3.1 System Monitoring and Data Collection

The data collection module is responsible for observing real host activity. It captures process actions, file access, and network activity at fixed intervals and converts them into a common event format. The output is a clean stream of normalized events that can be consumed by the graph builder.

The figure shows the flow from system sources through the collector into a normalized event stream.

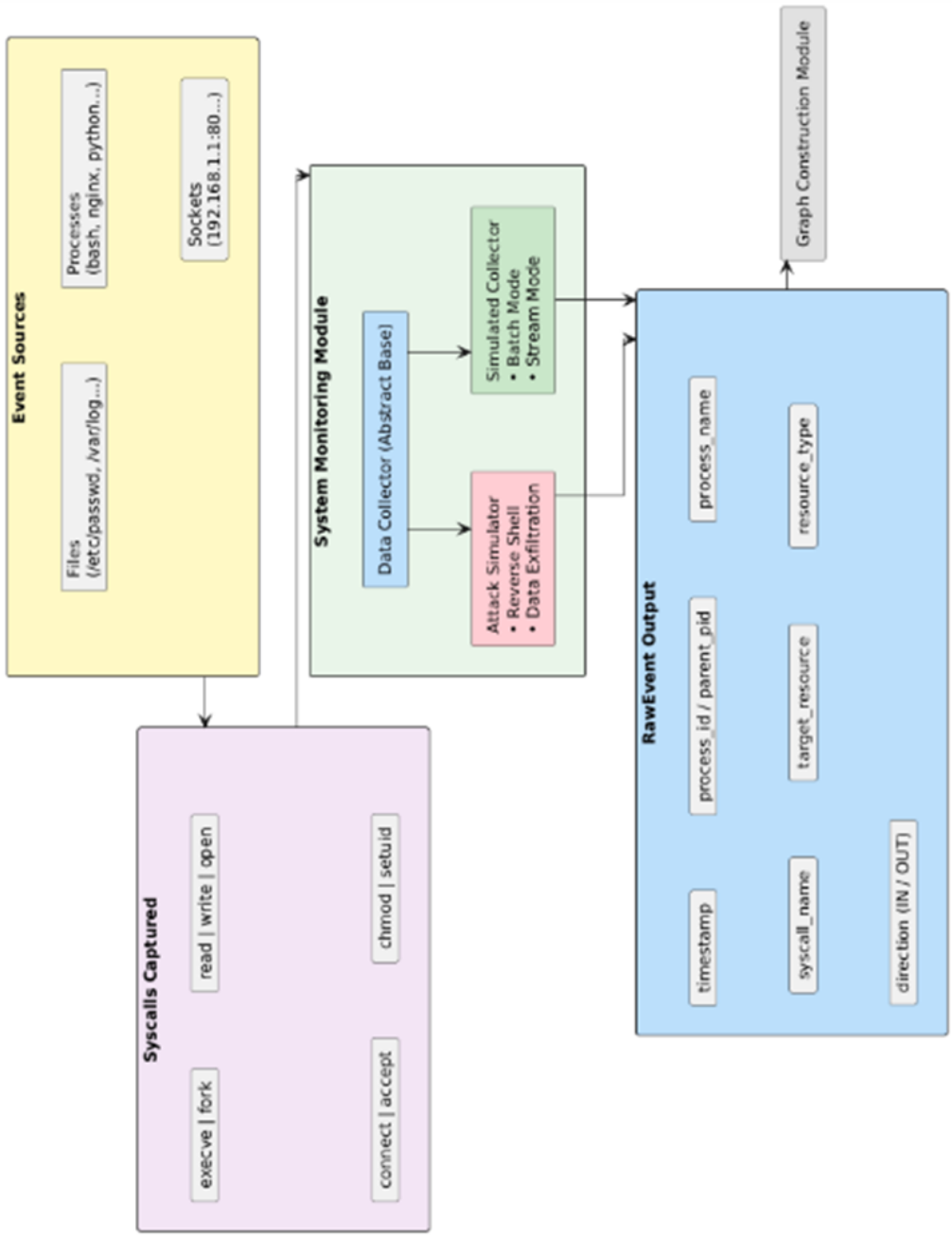

Figure 3.2: System monitoring pipeline that produces normalized raw events.

This module establishes the raw event stream used by all downstream stages.

3.3.2 Behavioral Graph Construction

The graph construction module groups events into time windows and converts each window into a behavioral graph. Nodes represent processes, files, and sockets. Edges

represent interactions such as executes, reads, writes, spawns, and connects. Node features are derived from type and connectivity statistics so the model can learn structure as well as activity intensity.

The figure outlines how events are grouped into windows and converted into nodes, edges, and feature vectors.

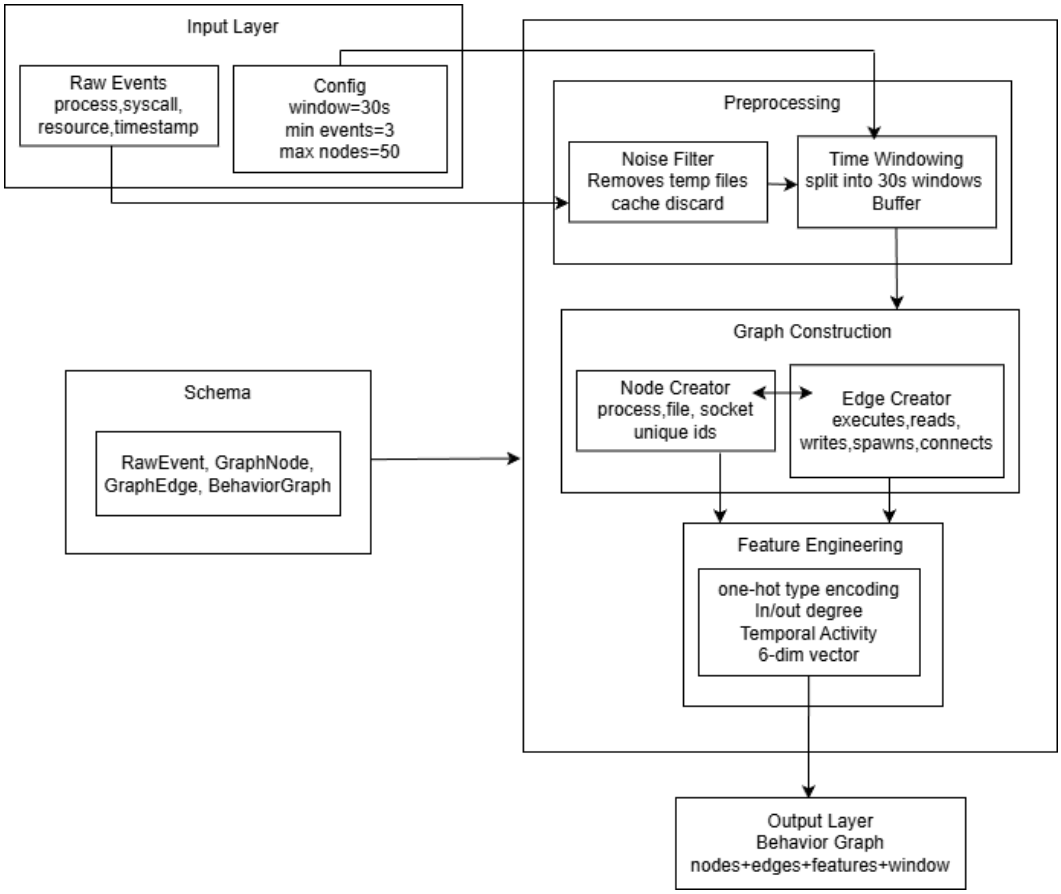

Figure 3.3: Behavioral graph construction workflow with preprocessing and feature creation.

It shows the transformation from events to structured graphs used for learning.

3.3.3 Graph Autoencoder Training and Baseline

The learning module trains a graph autoencoder only on normal graphs. The encoder compresses each graph into a latent representation using GCN layers, while the decoder reconstructs the adjacency structure. Reconstruction error becomes the anomaly signal. The training output includes the mean and standard deviation of errors, which define the baseline threshold for detection.

The figure highlights the encoder–decoder flow and how the baseline statistics are derived from reconstruction error.

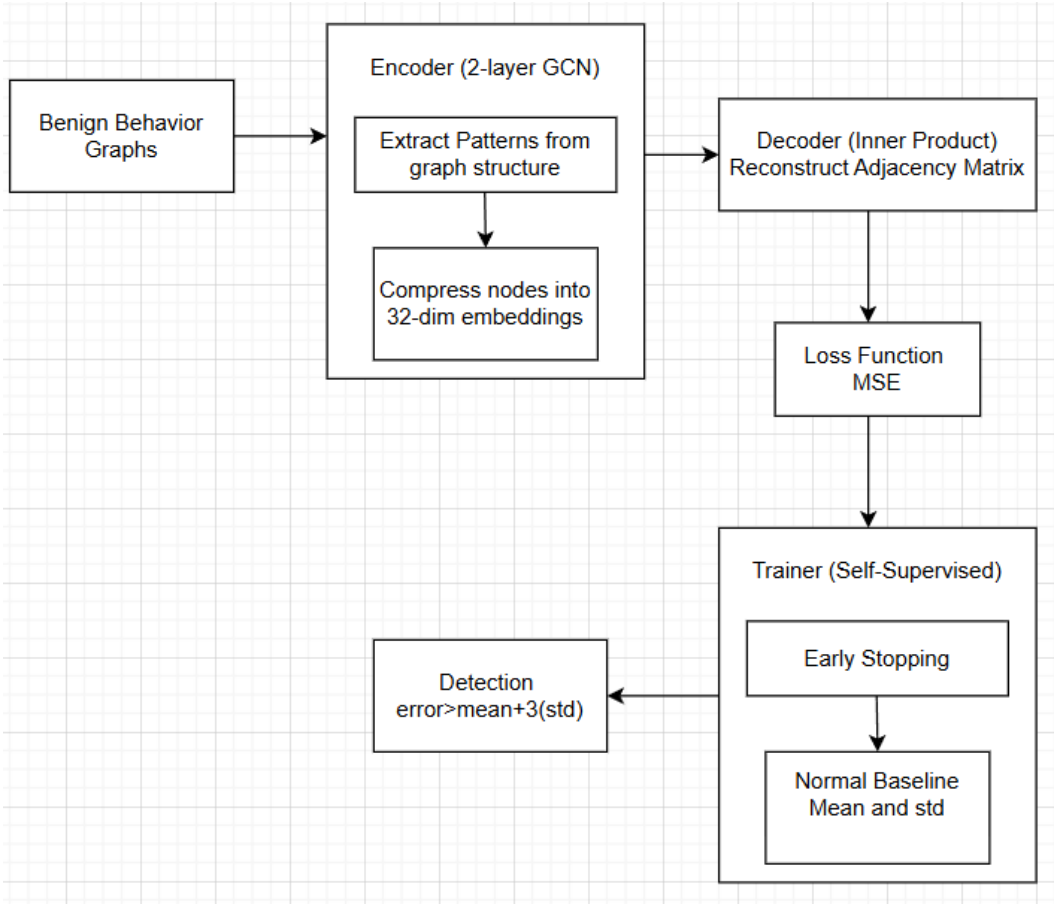

Figure 3.4: Graph autoencoder training pipeline and baseline statistics computation.

This diagram clarifies how the baseline threshold is computed from normal data.

3.3.4 Detection and Alerting

During detection, each new graph is passed through the trained autoencoder to compute a reconstruction score. If the score exceeds the learned threshold, the graph is flagged as anomalous and an alert is created with severity information. This module also records detection results for reporting and analysis.

3.3.5 Attack Simulation and Evaluation

To validate detection behavior, the system includes an attack simulator that generates synthetic streams for common attack categories. These events are merged with normal activity and converted to graphs so the detector can be evaluated under controlled conditions.

The figure shows how normal and simulated attack streams are combined before graph conversion and scoring.

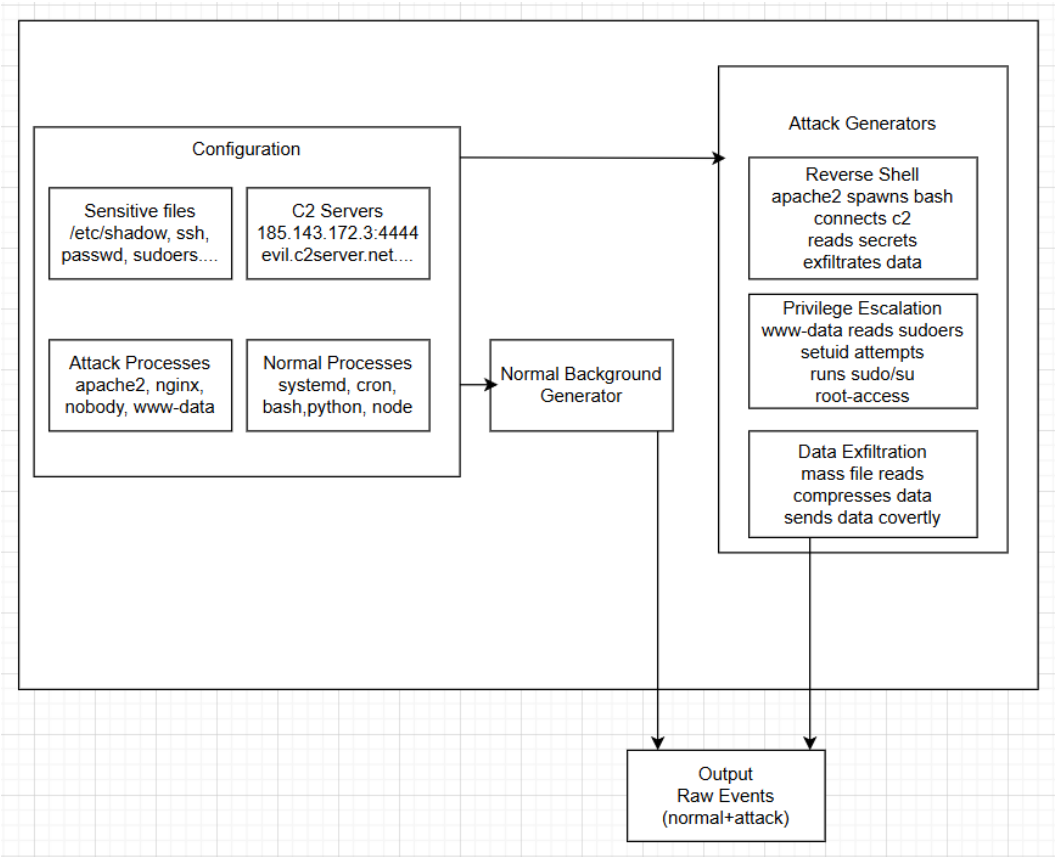

Figure 3.5: Attack simulation pipeline for controlled evaluation.

The simulated streams ensure repeatable and measurable detection tests.

3.3.6 Visualization and Reporting

The visualization module provides graph views and summary plots that allow a reviewer to compare normal and abnormal behavior. These views support qualitative validation of detection outcomes and help explain why a graph was flagged.

3.4 End-to-End Data Flow

The system executes the following sequence:

1. Capture raw system events during normal operation.
2. Convert events into time-windowed behavioral graphs.
3. Train the graph autoencoder on normal graphs only.
4. Compute baseline statistics for anomaly thresholds.
5. Score new graphs and generate alerts if thresholds are exceeded.

6. Inject simulated attacks and evaluate detection performance.
7. Export logs and visual summaries for reporting.

3.5 Summary

This chapter redefines the system design around the major module diagrams. Each figure corresponds to a pipeline stage, from event capture through graph learning to detection and evaluation, providing a consistent and visually grounded description of the full system.

REFERENCES

BIBLIOGRAPHY

[1] Albert Bifet, João Gama, and Bernhard Pfahringer. Streaming machine learning: Challenges and opportunities. In *Proceedings of the 2018 ACM SIGMOD Conference on Knowledge Discovery and Data Mining Workshop*, pages 1–4, 2018.

[2] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 1597–1607, 2020.

[3] Gabriele Demetrio, Mauro Valenza, Gabriele Costa, and Giovanni Lagorio. Adversarial machine learning at scale. In *Proceedings of the 2020 IEEE International Conference on Artificial Intelligence and Applications (CIS)*, pages 1423–1428, 2020.

[4] Dorothy E. Denning. An intrusion-detection model. *IEEE Transactions on Software Engineering*, SE-13(2):222–232, 1987.

[5] Yushun Dong, Milo Schuster, and Chenxu Zhang. A comprehensive survey on knowledge graphs with application to digital libraries. In *Library Trends*, volume 70, pages 15–38, 2021.

[6] Min Du, Feifei Li, Guofei Zheng, and Vivek Srikumar. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1285–1298, 2017.

[7] Sudeep Goel, Carla Robson, Matteo Poletto, and Cédric Fournet. Reconstructing system call context for forensic intrusion analysis. In *Proceedings of the 10th European Symposium on Research in Computer Security (ESORICS)*, pages 193–211, 2005.

[8] Palash Goyal, Nitin Kamra, Xinran He, and Yushun Liu. Dyngem: Deep embedding method for dynamic graphs. In *Proceedings of the 2018 International Workshop on Mining and Learning with Graphs*, 2018.

[9] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 1024–1034, 2017.

[10] Kaiming He, Xinlei Chen, Saining Xie, Yanghui Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the 2022 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[11] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9729–9738, 2020.

[12] Geoffrey E. Hinton and Ruslan R. Salakhutdinov. Reducing the dimensionality of data with neural networks. In *Science*, volume 313, pages 504–507, 2006.

[13] Mohammad Eusuff Hoque and Muhammad A. Mukit. An implementation of intrusion detection system using genetic algorithm. *arXiv preprint arXiv:1210.6351*, 2012.

[14] Will Huang and Jack W. Stokes. Mtnet: A multi-task neural network for dynamic malware behavior detection. In *Proceedings of the International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*, pages 399–418, 2015.

[15] Thomas Kipf and Max Welling. Variational graph auto-encoders. In *NIPS Workshop on Bayesian Deep Learning*, 2016.

[16] Thomas Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.

[17] Clemens Kolbitsch, Fredrik Valeur, and Giovanni Vigna. Dynamic behavior analysis of malware using call graphs. In *Proceedings of the 2009 6th International Workshop on Detection of Intrusions and Malware and Vulnerability Assessment (DIMVA)*, pages 39–58, 2009.

[18] Anukool Lakhina, Mark Crovella, and Christophe Diot. Diagnosing network-wide traffic anomalies. In *Proceedings of the 2004 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications*, pages 219–230, 2004.

[19] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining*, pages 413–422, 2008.

[20] Donald L. Lough. *A taxonomy of computer attacks with applications to wireless networks*. PhD thesis, Virginia Tech, 2001.

[21] Robert Luh, Sebastian Völker, Daniel Koch, and Michael Gerzon. Turbine: Heterogeneous heterogeneous graphs for zero-day attack detection. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy (S&P)*, pages 1234–1250, 2022.

[22] Tom M. Mitchell. Machine learning and data mining. *Communications of the ACM*, 40(11):24–26, 1997.

[23] Mark E. J. Newman. The structure and function of complex networks. *SIAM Review*, 45(2):167–256, 2003.

[24] Yushun Pei, Alberto Garcia-Duran, Dennis Krompass, and Brian Hoover. Heterogeneous graph neural network for rating prediction. In *Proceedings of the 2021 IEEE International Conference on Data Mining (ICDM)*, pages 1266–1271, 2021.

[25] Bernhard Schölkopf, John C. Platt, John Shawe-Taylor, Alex J. Smola, and Robert C. Williamson. Estimating the support of a high-dimensional distribution. *Neural Computation*, 13(7):1443–1471, 2001.

[26] Stacey Truex, Nathalie Baracaldo, Ali Anwar, et al. A hybrid approach to privacy-preserving federated learning. *Proceedings of the 12th ACM Workshop on Artificial Intelligence and Security*, pages 1–11, 2019.

[27] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. In *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2017.

[28] Xiao Wang, Xiangnan He, Yujia Cao, Meng Liu, and Tat-Seng Chua. Kgat: Knowledge graph attention network for recommendation. In *Proceedings of the 25th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pages 950–958, 2019.

[29] Stanley Wasserman and Katherine Faust. *Social network analysis: Methods and applications*. Cambridge University Press, 1994.

[30] Duncan J. Watts and Steven H. Strogatz. Collective dynamics of 'small-world' networks. *Nature*, 393(6684):440–442, 1998.

[31] Le Zhao, Yuansheng Zhang, Yanan Li, and Gang Liu. Graphbat: Graph-based behavioral anomaly detection for zero-day attacks. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (S&P)*, pages 1501–1517, 2023.

[32] Lichao Zhou, Yushun Yang, Xiang Ren, Fei Wu, and Yufeng Luan. Dynamic graph convolutional networks for learning on point clouds. In *Proceedings of the 2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9239–9248, 2021.

[33] Dipak Zhu, Han Xia, Jose Leme, Wei Qin, and Bo Yuan. Graph structure learning for robust graph neural networks. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pages 2149–2159, 2021.